NUR A - Assignment 2

Taotao Yang (s4866835)

Dated: March 14, 2026

1 Satellite galaxies around a massive central

The number density is given by:

$$n(x) = A\langle N_{\text{sat}}\rangle \left(\frac{x}{b}\right)^{a-3} \exp\left[-\left(\frac{x}{b}\right)^{c}\right] \tag{1}$$

The general form of the 3D spherical integral for this question is given as:

$$\iiint_V n(x)dV = \langle N_{\text{sat}}\rangle \tag{2}$$

Full code listed in appendix, with individual code snippets listed per question.

1.1 (1a) Determining the normalization

Let us consider integration of a sphere such that, when we integrate along some radius r, we build up the total volume by summing the thin spherical shells with a surface area S of:

$$S = 4\pi r^2 \Rightarrow V = \int S dr = \int 4\pi r^2 dr \Rightarrow dV = 4\pi r^2 dr \tag{3}$$

Since the number density $n(x)$ contains x that is directly related to r, we see that the dV element is esstially proportional to x. Let us consider some geometric units where $r_{\text{vir}} = 1$, we therefore have $x = r$:

$$dV = 4\pi r^2 dr \ ; \ x = \frac{r}{r_{\text{vir}}} = r \Rightarrow \frac{dr}{dx} = 1 \Rightarrow dV = 4\pi x^2 dx \tag{4}$$

Now, revisiting the general form of 3D spherical integral with a non-zero N_{sat}, we get:

$$4\pi \int n(x)x^2 dx = \langle N_{\text{sat}}\rangle \Rightarrow 4\pi \int \left(\frac{x}{b}\right)^{a-3} \exp\left[-\left(\frac{x}{b}\right)^{c}\right] x^2 dx = \frac{1}{A} \tag{5}$$

Obviously, we see a pitfall in this evaluation as zero value on the left hand side would result in infinity for A, we therefore must make a special case for $x = 0$. Physically, there shouldn't be any satellites when we are trying to find them within zero radius. Therefore, we manually define $n(0) = 0$ in functions. This enables us to find A from x.

For the numerical integrator, we employ Romberg integration routine as outlined in lecture slides (Lecture 4, p.15). Not only because Romberg routine is highly analogus to Richardson's

extrapolation method, it is also demonstrated in previous assignment that the Neville's algorithm produce highly accurate results. Since Nevill's algorithm is analogus to Romberg's integration routine, we can build up a table or matrix to iteratively refine towards the final result. That is, in a comparatively small number of iterations, we could efficiently reach reliable and accurate results.

In practice, we noticed that using the default provided order of two is insufficient in producing a reliable result, we therefore raised the order to a good number of ten to balance between efficiency and accuracy. And unlike the previous implementation of Neville's algorithm in previouos assignment, we favour inplace replacement of the results table/matrix, such that only the latest iteration 1-d array is kept. We obtain A from the given parameter values for $a = 2.4, b = 0.25, c = 1.6, x_{\max} = 5, \langle N_{\text{sat}} \rangle = 100$:

$$4\pi \int_0^5 (4x)^{-0.6} \exp\left[-(4x)^{1.6}\right] x^2 dx = \frac{1}{A} \tag{6}$$

The result is given below:

Normalization constant A
9.19479577651

Table 1: Normalization constant of the satellite galaxy profile.

1.2 (1a) Code for the integrator

```python
# INTG - START


# Below we provide a template for romberg integration
# You can implement this
# or use another integration method based on some form of Richardson extrapolation
def romberg_integrator(
    func: callable, bounds: tuple, order: int = 5, err: bool = False, args: tuple = ()
) -> float:
    """
    Romberg integration method

    Parameters
    ----------
    func : callable
        Function to integrate.
    bounds : tuple
        Lower- and upper bound for integration.
    order : int, optional
        Order of the integration.
        The default is 5.
    err : bool, optional
        Whether to retun first error estimate.
        The default is False.
    args : tuple, optional
        Arguments to be passed to func.
        The default is ().

    Returns
    -------
```

```python
    float
        Value of the integral. If err=True, returns the tuple
        (value, err), with err a first estimate of the (relative)
        error.
    """
    # local assignment
    a, b = bounds
    h = b - a
    fa = func(a, *args)
    fb = func(b, *args)

    # romberg - init
    r = np.zeros(order)
    # get r_0
    r[0] = (0.5 * h) * (fa + fb)

    # romberg - fill 1st col
    Np = 1
    for i in range(1, order):
        # new local assignment
        the_sum = 0.0
        delta = h
        h /= 2.0
        x = a + h

        for _ in range(Np):
            the_sum += func(x, *args)
            x += delta

        r[i] = 0.5 * (r[i - 1] + delta * the_sum)
        Np *= 2

    # romberg <-> neville
    # reset assignmnet
    Np = 1
    # loop through col
    for i in range(1, order):
        Np *= 4
        # loop through row
        for j in range(0, order - i):
            r[j] = (Np * r[j + 1] - r[j]) / (Np - 1)
    # get r[0] as best estimate for integral
    result = r[0]

    # errors
    if err:
        if order > 1:
            error = np.abs(r[1] - r[0])
        else:
            error = 0.0
        return result, error

    return result


# INTG - END
```

1.3 (1b) Sampling the satellite distribution

Describe the sampling method used (e.g. rejection sampling). Explain how the random number generator was implemented and how its randomness was tested.

The probability distribution in terms of number of galaxies $N(x)$ at radius x is given as:

$$p(x)dx = \frac{N(x)dx}{\langle N_{\text{sat}} \rangle} \tag{7}$$

Recall in part (a) that we evaluated the total number of galaxies via:

$$4\pi \int n(x)x^2 dx = \langle N_{\text{sat}} \rangle \Rightarrow 4\pi n(x)x^2 = N(x) \tag{8}$$

We can therefore rewrite the probability distribution as:

$$p(x)dx = \frac{4\pi n(x)x^2 dx}{\langle N_{\text{sat}} \rangle} \Rightarrow p(x) = \frac{4\pi n(x)x^2}{\langle N_{\text{sat}} \rangle} \tag{9}$$

Therefore, by envoking the relation in part (a) regarding the number density within the range of radius x at $x \in [0, 5)$, we get:

$$p(x) = \frac{4\pi n(x)x^2}{\langle N_{\text{sat}} \rangle} = \frac{4\pi n(x)x^2}{\int 4\pi n(x)x^2 dx} \tag{10}$$

Effectively, we see that the probability distribution is a normalized distribution, as the max probability occurs when all x in range $x \in [0, 5)$ is sampled.

For this part of the question, we implemented slice sampling as outlined in lecture slides (Lecture 5, p.16). We choose to implement slice sampling over other methods such as rejection sampling because it requires minimal supervision in its execution. Additionally, since we are dealing with a normalized probability distribution, we expect the uniform distribtion needed for slice sampling could very conveniently lead to a y value, where we could randomly try the neighboring space around its horizon to find the appropriate left-right bound for next iteration. Therefore, we can always guarentee that slice sampling would be successful in generating appropriate samples, especailly that the distribution $p(x)$ is expected to be continuous.

To introduce randomness for this operation, we created a custom random number generator that utilized both 64-bit XOR shift (XOR) and Multiply with Carry (MWC) methods as outlined in lecture slides (Lecture 5, p.14). We first create component functions that handles the operations at each iteration. For the XOR method, we first define a general binary mask to confine the results to 64 bits. The XOR function then moves each element is the state with a predefined bit-wise operation. In this specfic case, we move every element right by 21 bits, then left by 35 bits, and finally right by 4 bits. For the MWC method, we find the lowest and highest bits with each given state, where an updated state is then generated via multiplication with a given number of 4294957665.

At this time, these individual operations produces two states, both 64 bit. We then implement a wrapper fuction to return a random number generator value by merging the states together. This is done by taking the lower 32 bits from MWC and the full 64-bit XOR output through an AND gate. That is, we have now the building blocks for handling one iteration of random

number generation with state updates. We therefore create a custom class to parse some initial seed to create the initial states for the two random number generation methods at zero iteration. In this class, we obtain two different results, one being the combined XOR and MWC result as an integer $i \in [0, 2^{64} - 1]$, the other as a float $f \in [0, 1]$. We include the following figure to demonstrate the probability distribution of 10000 samples generated by the custom random number generator across 100 bins.

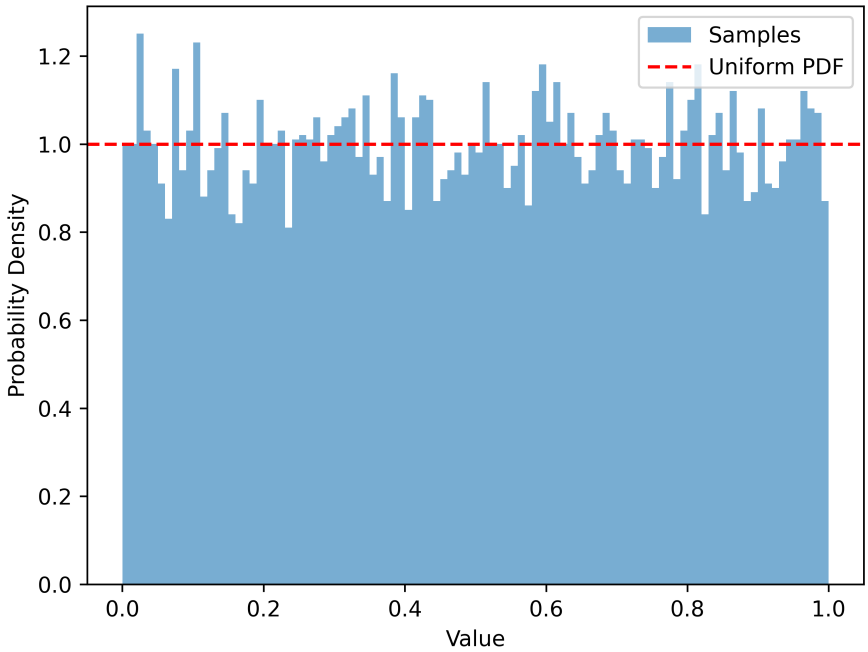

Figure 1: Comparison of custom random number generator output with unity.

We see that the custom random number generator does not perfectly produce a uniform distribution. However, by visual inspection, this is an acceptable random number generator for the purpose of this problem. It is important to note the difference between state and seed. The seed is the initial configuration that we define in order to create the initial states. The wrapper function use this common seed to produce the initial states, with which, the XOR and MWC methods iteratively update the states through their operations. That is, since we never re-seed the random number generators, we do not run the risk of these methods repeating itself intentionally.

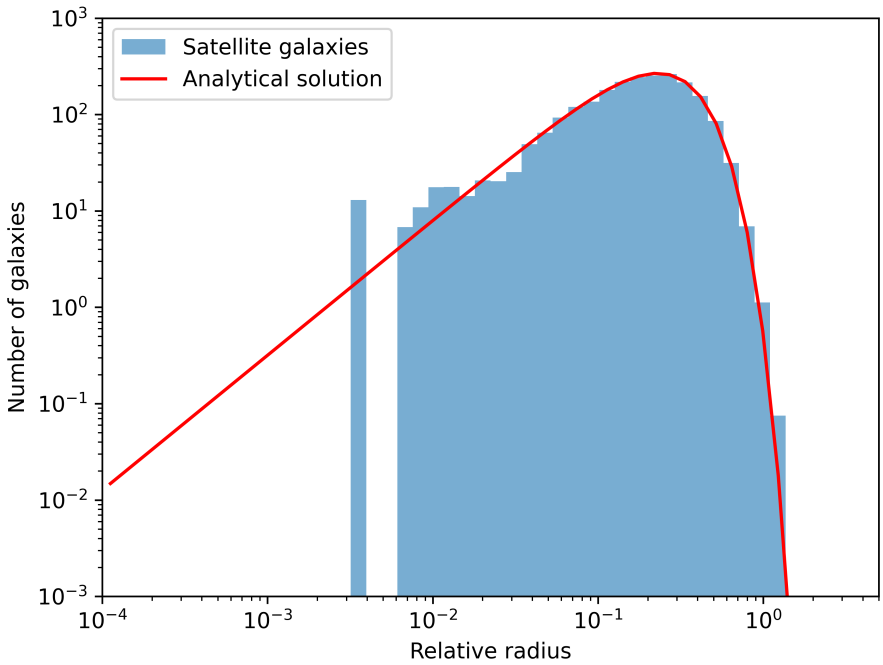

Figure 2: Comparison between the analytical satellite distribution $N(x)$ and the histogram of 10000 sampled satellite galaxies in logarithmic bins.

Regarding the results as shown above, we see that the sampled results match with analytic solution as expected. This match is increasinly visible as we increse the number of bins used for plot generation. The interesting bit occurs at low relative radius, where in the range of $x \in [10^{-4}, 10^{-2}]$, we observe very sparse amount of samples, this is consistent with slice sampleing method that we have implmented, as the small random step we introduced in our function for finding the left and right bounds may not be sufficiently small to account for very small x values. However, it is reasonable to believe that the results are consistent with peak values in the range of $x \in [10^{-2}, 10^{0}]$.

1.4 (1b) Code for rng

```
# RNG - START

# local const
MASK_64 = (1 << 64) - 1
MASK_32 = (1 << 32) - 1
XOR_A = 21
XOR_B = 35
XOR_C = 4
MWC_A = 4294957665
NUM_2_64 = 2**64


def xor_ops(state: int) -> int:
```

```python
    """Do 64-bit XOR shift for rng gen"""
    # do bit-wise ops through XOR-shift
    # 1) x = x^(x>>a)
    # 2) x = x^(x<<b)
    # 3) x = x^(x>>c)
    # we reapply 64 bit mask at every op to ensure consistency
    state ^= state >> XOR_A
    state &= MASK_64
    state ^= state << XOR_B
    state &= MASK_64
    state ^= state >> XOR_C
    state &= MASK_64

    return state


def mwc_ops(state: int) -> int:
    """Do multiply with carry with base 2**32"""
    # find lowest and highest bits in this state
    lowest = state & MASK_32
    highest = state >> 32
    # combine into new state
    # apply 32-bit mask for consistency
    state = MWC_A * lowest + highest
    state &= MASK_64

    return state


def rng_gen(state_xor: int, state_mwc: int) -> tuple[int, int, int]:
    """Evolve rng with xor and mwc methods"""
    # get next state from current state
    nxt_state_xor = xor_ops(state_xor)
    nxt_state_mwc = mwc_ops(state_mwc)

    # use lower 32 bits for mwc
    nxt_state_mwc_32 = nxt_state_mwc & MASK_32

    # merge results
    rng_val = nxt_state_xor ^ nxt_state_mwc_32

    return nxt_state_xor, nxt_state_mwc, rng_val


class NURARNG:
    """Custom RNG for NURA assignment"""

    # seed vs state
    # seed -> something you start with, it doesn't change
    # state -> computed based off of seed, it changes per niter

    def __init__(self, seed: int = 42) -> None:
        """Seed assignment init"""
        # avoid zero seed for xor
        if seed == 0:
            seed = 42

        seed &= MASK_64
        self.state_xor = seed
        self.state_mwc = seed

    def nxt_iter(self) -> int:
```

```python
        """Get to next iter, given current iter results, return rng_val"""
        nxt_state_xor, nxt_state_mwc, rng_val = rng_gen(self.state_xor, self.state_mwc)
        self.state_xor = nxt_state_xor
        self.state_mwc = nxt_state_mwc
        return rng_val

    def rand(self) -> float:
        """Return normalized rng_val similar to np.random.rand()"""
        return self.nxt_iter() / NUM_2_64


rng = NURARNG()


def plot_rng_uniformity(n_bins: int = 100, n_samples: int = 10000) -> None:
    """Plot rng results"""
    # sample gen
    samples = np.array([rng.rand() for _ in range(n_samples)])

    fig, ax = plt.subplots()
    ax.hist(samples, bins=n_bins, density=True, alpha=0.6, label="Samples")
    ax.axhline(1.0, color="r", linestyle="--", label="Uniform PDF")

    ax.set_xlabel("Value")
    ax.set_ylabel("Probability Density")
    ax.legend()

    fig.savefig("./plots/a2q1_rng_uniformity_test.png", dpi=600)
    plt.close(fig)


# RNG - END
```

1.5 (1b) Code for sampling

```python
# SAMP - START


def sampler(
    dist: callable,
    min_val: float,
    max_val: float,
    Nsamples: int,
    args: tuple = (),
) -> np.ndarray:
    """
    Sample a distribution using sampling method of your choice - slice sampling

    Parameters
    dist : callable
        Distribution to sample
    min_val :
        Minimum value for sampling
    max_val : float
        Maximum value for sampling
    Nsamples : int
        Number of samples
    args : tuple, optional
        Arguments of the distribution to sample, passed as args to dist

    Returns
```

```python
    -------
    sample: ndarray
        Values sampled from dist, shape (Nsamples,)
    """
    # choose initial x with dist(x) > 0 -> say we start with a mid point x0
    x = 0.5 * (min_val + max_val)
    # endure initial dist(x0) > 0 -> loop until it is
    while dist(x, *args) <= 0.0:
        x = min_val + (max_val - min_val) * rng.rand()

    # init samples
    samples = []
    # set step size for horizon tracing
    delta = 0.0001 * (max_val - min_val)

    # generate y ~ U(0, dist(x)) = U(0, 1)*dist(x)
    for _ in range(Nsamples):
        px = dist(x, *args)
        y = px * rng.rand()

        # get horizon bound x0, x1
        x0, x1 = x, x
        while x0 > min_val and dist(x0, *args) > y:
            x0 -= delta
        while x1 < max_val and dist(x1, *args) > y:
            x1 += delta

        # get the new sample
        while True:
            x_sample = x0 + (x1 - x0) * rng.rand()
            if dist(x_sample, *args) >= y:
                break
            # scale down to region
            if x_sample < x:
                x0 = x_sample
            else:
                x1 = x_sample

        samples.append(x_sample)

        # update x and goto next iter
        x = x_sample

    return np.array(samples)


# SAMP - END
```

1.6 (1c) Selecting and sorting satellites

Describe the selection of 100 galaxies (equal probability, no duplicates, no rejection). Explain the sorting algorithm you used.

We implement heap sort as outlined in lecture slides (Lecture 6, p.14) for this part of the problem. We choose heap sort such that we could utilize resursion for sorting. That is, with heap sort always at $O \log n$, we can obtain a sorted array with large length reasonably fast. Additionally, we could do this completely inplace. This is not only because that we need to rebuild a heap after each swap, but also that we could save a little bit of computational resources by doing so. By implementing sorting algorithm on the samples, we now arrange these sampled galaxies in an orderly fashion. And because the custom random number generator is never re-seeded, we do not expect a galaxy with the same configuration to appear in this sorted array of samples.

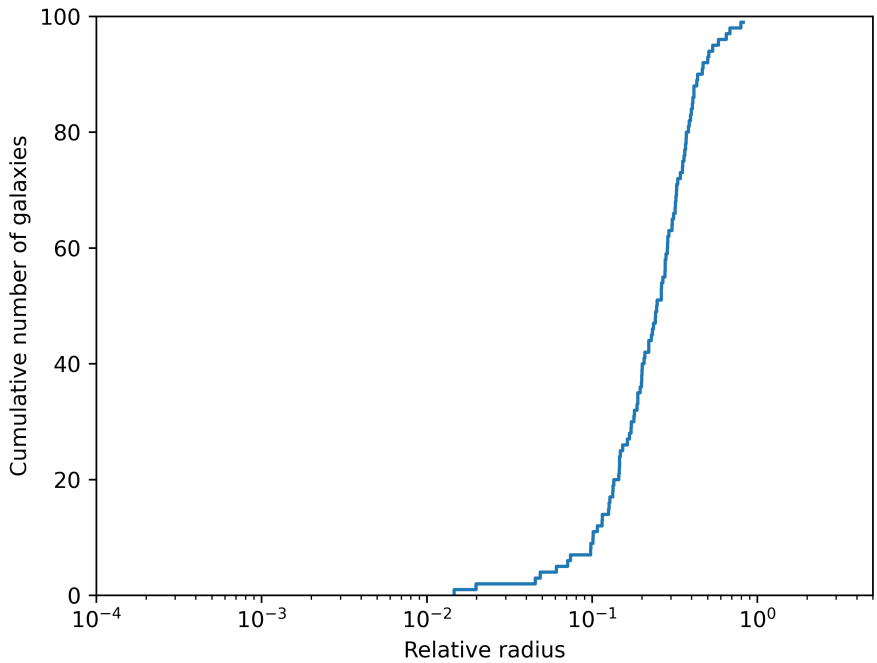

Figure 3: Cumulative number of selected galaxies as a function of relative radius.

In the figure above, we demonstrate the cumulative number of selected galaxies. We observe that at lower relative radius, the number of galaxies chosen is low. This is expected behavior, as the number of galaxies sampled at low relative readius is marginal comparing to a higher relative radius regime. We see that the majority of galaxies are picked as relative radius approaches $x \in [10^{-1}, 10^{0}]$, this coincides with the analytic solutions of galaxy number density. For choosing, we again introduce the custom random number generator. By looping over the sorted samples array, we can pick a random index and swap the sample with the last element of the sorted array. This follows Fisher-Yates shuffling scheme, such that, by reducing the range to loop over at each iteration, we have removed the already picked galaxy from the loot pool of the next

selection iteration. Since the custom random number generator is not a perfect one with uniform distribtion, we cannot strictly say that all galaxies are to be picked with equal probability, however, by translating the generated random number to integer index by multiplying with the current array length, we are effectively outputting a single integer that is reasonaly random.

1.7 (1c) Code for selection and sorting

```python
# SORT - START


def heap_ops(ary: np.ndarray, n: int, i: int) -> None:
    """Recursive heap sorting ops, build heap with size n"""
    # [2, 1, 5, 3, 7, 6, 8]
    # heap 0 1 2 3 4 5 6
    # elem 2 1 5 3 7 6 8

    # max heap at   =>  swap
    # 8                 5
    # 3   7             3   7
    # 2 1 6 5           2 1 6 /8

    # recursive
    # to max heap   =>  swap
    # 7                 5
    # 3   6             3   6
    # 2 1 5 /8          2 1 /7 /8

    # get max idx
    max_heap = i
    # get idx bounds - heap child of idx i
    l = 2 * i + 1
    r = 2 * i + 2

    # find max idx
    if n > l and ary[l] > ary[max_heap]:
        max_heap = l
    if n > r and ary[r] > ary[max_heap]:
        max_heap = r

    # swap
    if max_heap != i:
        ary[max_heap], ary[i] = ary[i], ary[max_heap]

        # recursive ops
        heap_ops(ary, n, max_heap)


def sort_array(
    arr: np.ndarray,
    inplace: bool = False,
) -> np.ndarray:
    """
    Sort a 1D array using a sorting algorithm of your choice - heap sort

    Parameters
    ----------
    arr : ndarray
        Input array to be sorted
    inplace : bool, optional
        If True, sort the array in-place
```

```
        If False, return a sorted copy

    Returns
    -------
    ary : ndarray
        Sorted array (same shape as arr)

    """
    # check inplace flag
    if inplace == False:
        ary = arr.copy()
    else:
        ary = arr

    # get array size
    n = len(ary)

    # init max heap
    for i in range((n // 2) - 1, -1, -1):
        # build heap
        heap_ops(ary, n, i)

    # get max element and swap
    for i in range(n - 1, 0, -1):
        # swap largest element with last
        ary[0], ary[i] = ary[i], ary[0]
        # build heap from current state
        heap_ops(ary, i, 0)

    return ary


def choice(arr: np.ndarray, size: int = 1) -> np.ndarray:
    """
    Choose given number of random elements from an array, without replacement.
    Fisher-Yates shuffling

    Parameters
    ----------
    arr : ndarray
        Array to shuffle
    size : int, optional
        Number of elements to pick from array
        The default is 1

    Returns
    -------
    chosen : ndarray
        Randomly chosen elements from arr, shape (size,)
    """
    ary = arr.copy()

    if size > len(ary):
        raise ValueError(
            "Invalid slice size for choice(), slcie size must be leq than array size."
        )

    # loop over idx
    for i in range(size):
        # get a nex idx from current through rng
        mod = int((len(ary) - i) * rng.rand())
```

```
        j = i + mod
        # swap
        ary[i], ary[j] = ary[j], ary[i]

    return ary[:size]


# SORT - END
```

1.8 (1d) Numerical derivative

The results of derivatives produced by analytical and numerical solutions are listed below:

Method	Value
Analytical	-0.625324485666
Numerical	-0.625324485666

Table 2: Derivative of the satellite number density at $x = 1$.

We implemented Ridders' method as shown in lecture slides (Lecture 5, p.8) for derivative calculation. For finite difference, we use a step size h for some function $f(x)$:

$$\delta = \frac{f(x+h) - f(x-h)}{2h} \tag{11}$$

We see that these two results are consistent to 12 figures. Physically, this is representative of the number density of galaxies decreasing as radius approaches the maximum. Recall that $x = r/r_{\text{vir}}$, as $x1$, we expect the number density to fall drastically, just as the negative value shown in the table above. In terms of error handling, we manually defined a 10^{-9} value of difference between the current and last estimation. And unlike the routines in previous parts of this question, we do need to use a matrix to store the extrapolation table, as this is the simplist to write down for indexing. In terms of parameters, we let the bounds decrease by a factor of 2 at each iteration, with max iterations being 5. We could raise the number of max iterations but the current output is consistent enough that it may not be necessary. The same can be said for the manually defined error tolerance, we could further reduce it if needed, such that this routine would continue to refine the estimation more and more.

1.9 (1d) Code for derivative calculation

```python
# DIFF - START


def dn_dx(x: float, A: float, Nsat: float, a: float, b: float, c: float) -> float:
    """
    Analytical derivative of number density provide

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float
```

```python
        Same type and shape as x. Derivative of number density of
        satellite galaxies at given radius x.
    """
    # f(x) = A * Nsat * (x / b) ** (a - 3) * np.exp(-((x / b) ** c))
    # analytical solution
    top = (
        A
        * Nsat
        * b**3
        * (x / b) ** a
        * (c * (x / b) ** c - a + 3)
        * np.exp(-((x / b) ** c))
    )
    bot = x**4
    return -(top / bot)


def finite_difference(func: callable, x: float, h: float) -> float:
    """
    A building block to compute derivative using finite differences

    Parameters
    ----------
    func : callable
        Function to differentiate
    x : float
        Value(s) to evaluate derivative at
    h : float
        Step size for finite difference

    Returns
    -------
    dy : float
        Derivative at x
    """
    # return central difference
    return (func(x + h) - func(x - h)) / (2 * h)


def compute_derivative(
    func: callable,
    x: float,
    h_init: float,
    # For Ridders use parameters below:
    d: float = 2.0,
    eps: float = 1e-9,
    max_iters: int = 5,
) -> float:
    """
    Function to compute derivative

    Parameters
    ----------
    func : callable
        Function to differentiate
    x : float
        Value(s) to evaluate derivative at
    h_init : float
        Initial step size for finite difference
    d : float
        Factor by which to decrease h_init every iteration, default = 2.0
    eps : float
```

```python
        Relative error, default at 1e-9
    max_iters : int
        Maximum number of iterations before exiting, default = 5

    Returns
    -------
    df : float
        Derivative at x
    """
    # local repo
    h = h_init
    d_mat = np.zeros((max_iters, max_iters))
    err_last = np.inf
    best_estimate = d_mat[0, 0]

    # fill 1st col
    for i in range(max_iters):
        d_mat[i, 0] = finite_difference(func, x, h)
        h /= d

    # fill the rest
    for j in range(1, max_iters):
        for i in range(max_iters - j):
            top = d ** (2 * j) * d_mat[i + 1, j - 1] - d_mat[i, j - 1]
            bot = d ** (2 * j) - 1
            d_mat[i, j] = top / bot

        # get err estimate
        err = abs((d_mat[0, j] - d_mat[0, j - 1]) / d_mat[0, j])
        if err < eps:
            return d_mat[0, j]
        if err > err_last * 1.2 and j > 1:
            return best_estimate
        # update
        best_estimate = d_mat[0, j]
        err_last = err

    return best_estimate


# DIFF - END
```

2 Heating and cooling in HII regions

2.1 (2a) Photoionization heating and radiative recombination cooling

We have the following two function for equalibrium calculation. The first is a simplified case that focus on HII regions with only one heating and cooling mechanism. We start with the heating produced by photoionization.

$$\Gamma_{\rm pe} = \alpha_{\rm B} n_{\rm H} n_{\rm e} \psi k_{\rm B} T_{\rm c} \tag{12}$$

The radiative recombination is given with:

$$\Lambda_{\rm rr} = \alpha_{\rm B} n_{\rm H} n_{\rm e} \langle E_{\rm rr} \rangle = \alpha_{\rm B} n_{\rm H} n_{\rm e} \left[0.684 - 0.0416 \ln \left(\frac{T_4}{Z^2} \right) \right] k_{\rm B} T \tag{13}$$

Therefore, the equbilibrium is located when $\Gamma_{\rm pe} = \Lambda_{\rm rr}$:

$$\Gamma_{\rm pe} - \Lambda_{\rm rr} = 0 \Rightarrow \alpha_{\rm B} n_{\rm H} n_{\rm e} \left(\psi k_{\rm B} T_{\rm c} - \left[0.684 - 0.0416 \ln \left(\frac{T_4}{Z^2} \right) \right] k_{\rm B} T \right) = 0 \tag{14}$$

We see that this equlirbium problem becomes a root finding proble,. That is, we can employ Brent's method as illustrated in lecture slides (Lecture 6, p.25) for root finding. We choose Brent's method over Newton-Raphson for the reliability of Brent's method in producing a root when the initial bracket for root estimation is properly set. That is, when the initial bracket is set to contain at least one root, we expect Brent's method to always converge at some iteration, mitigating the risk of divergence in Newton-Raphson method, where explicit analytical derivative calculations are needed. It is worth noting that having exactly one root within the bracket would produce the most reliable result.

Additionally, due to Brent's method calls for bisection and secant method when conditions are met, we could quickly iterate to find the appropriate root for some function. In implementation, Brent's method employs bisection algorithm to narrow down the bracket to a smaller region, before quadratic and secant method are deployed to refine the estimates. We calculated that the equilibrium tempearature $T \simeq 3.25 \times 10^4$ K, with absolute and relative error shown in the table below:

Temperature [K]	Absolute error	Relative error	Iterations
32539.1267375	7.276e-12	2.236e-16	29

Table 3: Equilibrium temperature for the simplified heating-cooling model.

We were able to locate this root estimation at iteration index 29. We designed the implementation of Brent's method so that it could run recursivly, with some pre-defined maximum iterations number as safety to prevent endless recursion. The general pipeline is outlined in the code listing for this problem, with full code listed in the appendix. In short, we employed a similar approach inspired by the random number generator in question 1, where we utilize states for iterative calculations. The stopping conditions for this implementation of Brent's method is set so that either: 1) an exact root is found, 2) maximum number of recursion is reached, or 3) a bracket width is refined to something smaller than the estimated tolerance. And in order to make sure that the bracket is consistently valid, we check and update the bracket at every iteration.

2.2 (2b) More realistic model

We parsed in the complex version of equlibrium function in part (b) of this question. The general form of equilibrium is given by:

$$(\Gamma_{\rm pe} + \Gamma_{\rm cr} + \Gamma_{\rm mhd}) - (\Lambda_{\rm rr} + \Lambda_{\rm ff}) = 0 \tag{15}$$

With the default values supplied by the problem, we perform root finding for this function with the same routine deployed for part (a). The results are shown in the table below:

Density $n_{\rm H}$ [cm^{-3}]	Equilibrium temperature [K]	Absolute error	Relative error	Iterations
10^{-4}	1.60647874937e+14	8.976e+07	5.588e-07	23
1	33861.2948595	3.036e-02	8.965e-07	53
10^{4}	10525.8859988	8.294e-03	7.880e-07	54

Table 4: Equilibrium temperature for different gas densities.

As we can see, for low gas density, the absolute error is highly pronounced, while relatively intermediate to high density gas produce more consistent and meaningful results. We show the equalibrium function evaluation for different $n_{\rm H}$ configuration in the figure below, alongside the roots found through Brent's method:

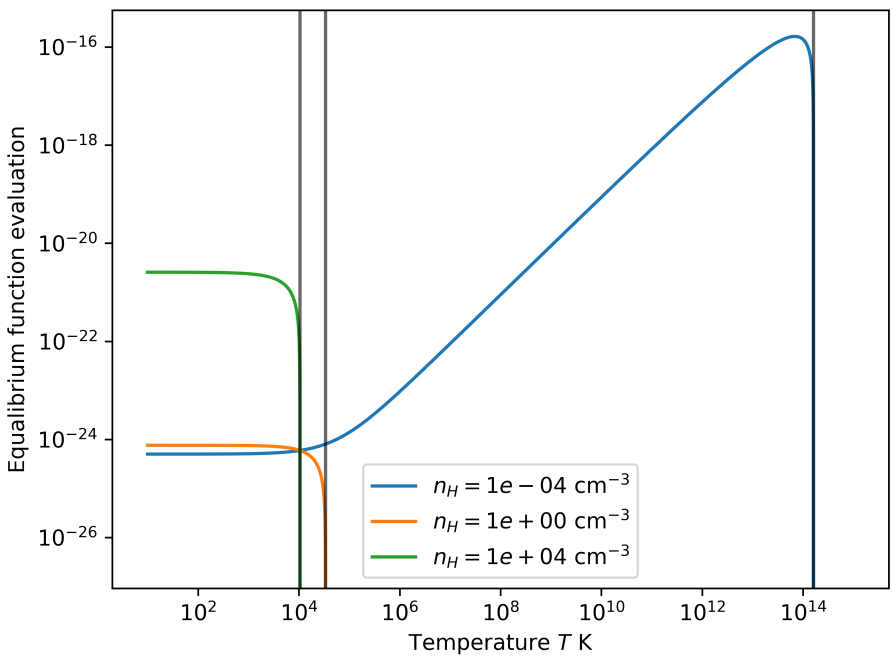

Figure 4: Equilirbium function evaluation with different $n_{\rm H}$ configuration. Black vertical lines indicate root found with Brent's method.

It is observed that, for these three different configurations of gas densities, our implementation of Brent's method converged within 60 iterations. We manually defined a maxmimum iteration number at 1000, the very wide initial bracket setting of $x \in [1, 1 \times 10^{15}]$ could add to the initial number of iterations, as repeated bisections were needed to narrow the bracket down to a reasonable width for secant and quadratic methods to kick in. By plotting $\Gamma - \Lambda$ against T on log space, we perform a sanity check for the calculated roots from Brent's method, where slop of $\Gamma - \Lambda$ approaches $-\infty$ as T apparoches the found root of zero crossing. This is also good for visualizing that, for the selected bracket, we are indeed dealing with excatly one root.

2.3 Code for Exercise 2

```python
# ROOT - START


def root_finder(
    func: callable,
    bracket: tuple[float, float],
    atol: float = 1e-6,
    rtol: float = 1e-6,
    max_iters: int = 1000,
) -> tuple[float, float, float]:
    """
    Find a root of a function - Brent's method

    Parameters
    ----------
    func : callable
        Function fn to find root of
    bracket : tuple
        Bracket for which to find root
    atol : float, optional
        Absolute tolerance.
        The default is 1e-6
    rtol : float, optional
        Relative tolerance.
        The default is 1e-6
    max_iters: int, optional
        Maximum number of iterations.
        The default is 100

    Returns
    -------
    root : float
        Approximate root
    aerr : float
        Absolute error - final bracket width
    rerr : float
        Relative error - final width / |fb|
    """
    # local repo
    a, b = bracket
    fa, fb = func(a), func(b)

    # ensure at least one root is present in bracket
    # ideally only one root present is the best case
    if fa * fb >= 0.0:
        raise ValueError(f"Cannot guarantee n_root >= 1 with set bracket {bracket}.")

    # set b as the best estimate between a and b
```

```python
    # must have abs(fa) > abs(fb)
    # no swap needed if equal scale
    if abs(fa) < abs(fb):
        a, b = b, a
        fa, fb = fb, fa

    # set initial c as a
    # at next iter c is current b
    c = a
    niter = max_iters

    # init new point and diff size
    d = 0.5 * (b - a)
    e = d

    # make current state
    state = (a, b, c, d, e, niter)

    def check_bracket(state: tuple) -> tuple:
        """Check if the current [c, b] bracket is still valid"""
        # unpack and get val
        a, b, c, d, e, niter = state
        fb, fc = func(b), func(c)
        # check bracket
        if fb * fc > 0.0:
            e = b - a
            # fallback to c = a to ensure valid bracket
            state = (a, b, a, d, e, niter)

        return state

    def check_best_estimate(state: tuple) -> tuple:
        """Best estimation should be x=b"""
        a, b, c, d, e, niter = state
        fb, fc = func(b), func(c)

        # set b as best estimate
        # if |fb| > |fc| -> c is better estimate now
        if abs(fb) > abs(fc):
            # set a_new -> b, b_new -> c, c_new -> a
            state = (b, c, a, d, e, niter)

        return state

    def find_new_point(state: tuple) -> tuple:
        """Calculate d from current state"""
        a, b, c, d, e, niter = state
        fa, fb, fc = func(a), func(b), func(c)

        # method select
        if c == a:
            # secant method
            d_new = b - fb * (b - a) / (fb - fa)
        else:
            # quadratic
            r, s, t = fb / fc, fb / fa, fa / fc
            p = s * (t * (r - t) * (c - b) - (1.0 - r) * (b - a))
            q = (t - 1.0) * (r - 1.0) * (s - 1.0)
            d_new = b + p / q

        state = (a, b, c, d_new, e, niter)

```

```python
        return state

    def check_d(state: tuple) -> tuple:
        """Check if d satisfy the conditions"""
        a, b, c, d, e, niter = state
        low = 0.25 * (3 * a + b)
        high = b
        h = abs(d - b)
        h_last = abs(e)
        # bisection conditions check
        if (low <= d <= high) and (h < 0.5 * h_last):
            # no bisection
            d_new = d
            e_new = d - b
        else:
            # use bisection
            d_new = 0.5 * (a + b)
            e_new = b - a

        return (a, b, c, d_new, e_new, niter)

    def are_we_there_yet(state: tuple) -> tuple:
        """Check if a root is found, otherwise update state"""
        a, b, c, d, e, niter = state
        fb, fd = func(b), func(d)

        # exit if new point is root
        if fd == 0.0:
            return state

        # update bracket
        if fb * fd < 0.0:
            a_new, b_new = b, d
        else:
            a_new, b_new = a, d

        # enforce b as the best estimate
        fa_new, fb_new = func(a_new), func(b_new)
        # if |fb| > |fa| -> a is better estimate -> swap
        if abs(fb_new) > abs(fa_new):
            a_new, b_new = b_new, a_new

        # update c_new = b
        c_new = b

        # update niter counter
        niter -= 1

        # rebuild state
        state = (a_new, b_new, c_new, d, e, niter)

        return state

    def check_convergence(state: tuple) -> tuple:
        """Recursion stop consition checker"""
        a, b, c, d, e, niter = state
        fb = func(b)

        # get current tolerance
        tol = atol + rtol * abs(b)
        aerr = abs(b - a)
        rerr = aerr
```

```python
        flag = False
        root = None

        # check convergence
        if (aerr < tol) or (niter == 0) or (fb == 0.0):
            # yes convergence
            flag = True
            root = b
            if b != 0.0:
                rerr = aerr / abs(b)

        # build output for main return
        result = (flag, niter, root, aerr, rerr)

        return result

    def evolve(state: tuple) -> tuple:
        """Recursivly do Brent's method to get the final state"""
        flag, niter, root, aerr, rerr = check_convergence(state)

        # exit if convergence achieved
        if flag == True:
            return root, aerr, rerr, max_iters - niter

        # main pipeline
        # validity check
        state = check_bracket(state)
        state = check_best_estimate(state)
        # find new point d
        state = find_new_point(state)
        # check if d is valid
        state = check_d(state)
        # evaluate and check
        state = are_we_there_yet(state)

        # recursive
        return evolve(state)

    return evolve(state)


# ROOT - END
```

3 Conclusions

We have implemented Heap Sort, Romberg intergration routine, and Brent's method for root finding for this problem set. From our assessments of code implementation choices and figures produced, we are able to produce consistent and reliable results within relatively short amount of time. We found that, similar with the previous problem set, it is helpful to visualize the algorithm on paper before implmentation. Some notable difficulties in coding up the routines in this problem set includes the initial implmentation of recursion for heap sorting, and writing out the steps needed for Brent's method. It is also possible that the current implementation may be applicable to extreme cases, where recursive runs may be halted due to resource concerns.

A Full Code Listing for Question 1

```python
"""
Scripts for assignment 2 question 1
"""

# imports
import numpy as np
import matplotlib.pyplot as plt

# ==========================================================================================
# RNG - START

# local const
MASK_64 = (1 << 64) - 1
MASK_32 = (1 << 32) - 1
XOR_A = 21
XOR_B = 35
XOR_C = 4
MWC_A = 4294957665
NUM_2_64 = 2**64


def xor_ops(state: int) -> int:
    """Do 64-bit XOR shift for rng gen"""
    # do bit-wise ops through XOR-shift
    # 1) x = x^(x>>a)
    # 2) x = x^(x<<b)
    # 3) x = x^(x>>c)
    # we reapply 64 bit mask at every op to ensure consistency
    state ^= state >> XOR_A
    state &= MASK_64
    state ^= state << XOR_B
    state &= MASK_64
    state ^= state >> XOR_C
    state &= MASK_64

    return state


def mwc_ops(state: int) -> int:
    """Do multiply with carry with base 2**32"""
    # find lowest and highest bits in this state
    lowest = state & MASK_32
    highest = state >> 32
    # combine into new state
    # apply 32-bit mask for consistency
    state = MWC_A * lowest + highest
    state &= MASK_64

    return state


def rng_gen(state_xor: int, state_mwc: int) -> tuple[int, int, int]:
    """Evolve rng with xor and mwc methods"""
    # get next state from current state
    nxt_state_xor = xor_ops(state_xor)
    nxt_state_mwc = mwc_ops(state_mwc)

    # use lower 32 bits for mwc
```

```python
    nxt_state_mwc_32 = nxt_state_mwc & MASK_32

    # merge results
    rng_val = nxt_state_xor ^ nxt_state_mwc_32

    return nxt_state_xor, nxt_state_mwc, rng_val


class NURARNG:
    """Custom RNG for NURA assignment"""

    # seed vs state
    # seed -> something you start with, it doesn't change
    # state -> computed based off of seed, it changes per niter

    def __init__(self, seed: int = 42) -> None:
        """Seed assignment init"""
        # avoid zero seed for xor
        if seed == 0:
            seed = 42

        seed &= MASK_64
        self.state_xor = seed
        self.state_mwc = seed

    def nxt_iter(self) -> int:
        """Get to next iter, given current iter results, return rng_val"""
        nxt_state_xor, nxt_state_mwc, rng_val = rng_gen(self.state_xor, self.state_mwc)
        self.state_xor = nxt_state_xor
        self.state_mwc = nxt_state_mwc
        return rng_val

    def rand(self) -> float:
        """Return normalized rng_val similar to np.random.rand()"""
        return self.nxt_iter() / NUM_2_64


rng = NURARNG()


def plot_rng_uniformity(n_bins: int = 100, n_samples: int = 10000) -> None:
    """Plot rng results"""
    # sample gen
    samples = np.array([rng.rand() for _ in range(n_samples)])

    fig, ax = plt.subplots()
    ax.hist(samples, bins=n_bins, density=True, alpha=0.6, label="Samples")
    ax.axhline(1.0, color="r", linestyle="--", label="Uniform PDF")

    ax.set_xlabel("Value")
    ax.set_ylabel("Probability Density")
    ax.legend()

    fig.savefig("./plots/a2q1_rng_uniformity_test.png", dpi=600)
    plt.close(fig)


# RNG - END
# ================================================================================ #
```

```python
# NDEN - START


def n(x: float, A: float, Nsat: float, a: float, b: float, c: float) -> float:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : float
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return A * Nsat * (x / b) ** (a - 3) * np.exp(-((x / b) ** c))


# NDEN - END
# ================================================================================ #
# INTG - START


# Below we provide a template for romberg integration
# You can implement this
# or use another integration method based on some form of Richardson extrapolation
def romberg_integrator(
    func: callable, bounds: tuple, order: int = 5, err: bool = False, args: tuple = ()
) -> float:
    """
    Romberg integration method

    Parameters
    ----------
    func : callable
        Function to integrate.
    bounds : tuple
        Lower- and upper bound for integration.
    order : int, optional
        Order of the integration.
        The default is 5.
    err : bool, optional
        Whether to retun first error estimate.
        The default is False.
    args : tuple, optional
        Arguments to be passed to func.
```

```python
        The default is ().

    Returns
    -------
    float
        Value of the integral. If err=True, returns the tuple
        (value, err), with err a first estimate of the (relative)
        error.
    """
    # local assignment
    a, b = bounds
    h = b - a
    fa = func(a, *args)
    fb = func(b, *args)

    # romberg - init
    r = np.zeros(order)
    # get r_0
    r[0] = (0.5 * h) * (fa + fb)

    # romberg - fill 1st col
    Np = 1
    for i in range(1, order):
        # new local assignment
        the_sum = 0.0
        delta = h
        h /= 2.0
        x = a + h

        for _ in range(Np):
            the_sum += func(x, *args)
            x += delta

        r[i] = 0.5 * (r[i - 1] + delta * the_sum)
        Np *= 2

    # romberg <-> neville
    # reset assignmnet
    Np = 1
    # loop through col
    for i in range(1, order):
        Np *= 4
        # loop through row
        for j in range(0, order - i):
            r[j] = (Np * r[j + 1] - r[j]) / (Np - 1)
    # get r[0] as best estimate for integral
    result = r[0]

    # errors
    if err:
        if order > 1:
            error = np.abs(r[1] - r[0])
        else:
            error = 0.0
        return result, error

    return result


# INTG - END
# ==============================================================================
```

```python
    #
# SAMP - START


def sampler(
    dist: callable,
    min_val: float,
    max_val: float,
    Nsamples: int,
    args: tuple = (),
) -> np.ndarray:
    """
    Sample a distribution using sampling method of your choice - slice sampling

    Parameters
    dist : callable
        Distribution to sample
    min_val :
        Minimum value for sampling
    max_val : float
        Maximum value for sampling
    Nsamples : int
        Number of samples
    args : tuple, optional
        Arguments of the distribution to sample, passed as args to dist

    Returns
    -------
    sample: ndarray
        Values sampled from dist, shape (Nsamples,)
    """
    # choose initial x with dist(x) > 0 -> say we start with a mid point x0
    x = 0.5 * (min_val + max_val)
    # endure initial dist(x0) > 0 -> loop until it is
    while dist(x, *args) <= 0.0:
        x = min_val + (max_val - min_val) * rng.rand()

    # init samples
    samples = []
    # set step size for horizon tracing
    delta = 0.0001 * (max_val - min_val)

    # generate y ~ U(0, dist(x)) = U(0, 1)*dist(x)
    for _ in range(Nsamples):
        px = dist(x, *args)
        y = px * rng.rand()

        # get horizon bound x0, x1
        x0, x1 = x, x
        while x0 > min_val and dist(x0, *args) > y:
            x0 -= delta
        while x1 < max_val and dist(x1, *args) > y:
            x1 += delta

        # get the new sample
        while True:
            x_sample = x0 + (x1 - x0) * rng.rand()
            if dist(x_sample, *args) >= y:
                break
            # scale down to region
            if x_sample < x:
                x0 = x_sample
```

```python
            else:
                x1 = x_sample

        samples.append(x_sample)

        # update x and goto next iter
        x = x_sample

    return np.array(samples)


# SAMP - END
# ==================================================================================
# SORT - START


def heap_ops(ary: np.ndarray, n: int, i: int) -> None:
    """Recursive heap sorting ops, build heap with size n"""
    # [2, 1, 5, 3, 7, 6, 8]
    # heap 0 1 2 3 4 5 6
    # elem 2 1 5 3 7 6 8

    # max heap at   =>  swap
    # 8                 5
    # 3   7             3   7
    # 2 1 6 5           2 1 6 /8

    # recursive
    # to max heap   =>  swap
    # 7                 5
    # 3   6             3   6
    # 2 1 5 /8          2 1 /7 /8

    # get max idx
    max_heap = i
    # get idx bounds - heap child of idx i
    l = 2 * i + 1
    r = 2 * i + 2

    # find max idx
    if n > l and ary[l] > ary[max_heap]:
        max_heap = l
    if n > r and ary[r] > ary[max_heap]:
        max_heap = r

    # swap
    if max_heap != i:
        ary[max_heap], ary[i] = ary[i], ary[max_heap]

        # recursive ops
        heap_ops(ary, n, max_heap)


def sort_array(
    arr: np.ndarray,
    inplace: bool = False,
) -> np.ndarray:
    """
    Sort a 1D array using a sorting algorithm of your choice - heap sort

```

```
    Parameters
    ----------
    arr : ndarray
        Input array to be sorted
    inplace : bool, optional
        If True, sort the array in-place
        If False, return a sorted copy

    Returns
    -------
    ary : ndarray
        Sorted array (same shape as arr)

    """
    # check inplace flag
    if inplace == False:
        ary = arr.copy()
    else:
        ary = arr

    # get array size
    n = len(ary)

    # init max heap
    for i in range((n // 2) - 1, -1, -1):
        # build heap
        heap_ops(ary, n, i)

    # get max element and swap
    for i in range(n - 1, 0, -1):
        # swap largest element with last
        ary[0], ary[i] = ary[i], ary[0]
        # build heap from current state
        heap_ops(ary, i, 0)

    return ary


def choice(arr: np.ndarray, size: int = 1) -> np.ndarray:
    """
    Choose given number of random elements from an array, without replacement.
    Fisher-Yates shuffling

    Parameters
    ----------
    arr : ndarray
        Array to shuffle
    size : int, optional
        Number of elements to pick from array
        The default is 1

    Returns
    -------
    chosen : ndarray
        Randomly chosen elements from arr, shape (size,)
    """
    ary = arr.copy()

    if size > len(ary):
        raise ValueError(
            "Invalid slice size for choice(), slcie size must be leq than array size."
```

```python
        )

    # loop over idx
    for i in range(size):
        # get a nex idx from current through rng
        mod = int((len(ary) - i) * rng.rand())
        j = i + mod
        # swap
        ary[i], ary[j] = ary[j], ary[i]

    return ary[:size]


# SORT - END
# ================================================================================
# DIFF - START


def dn_dx(x: float, A: float, Nsat: float, a: float, b: float, c: float) -> float:
    """
    Analytical derivative of number density provide

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float
        Same type and shape as x. Derivative of number density of
        satellite galaxies at given radius x.
    """
    # f(x) = A * Nsat * (x / b) ** (a - 3) * np.exp(-((x / b) ** c))
    # analytical solution
    top = (
        A
        * Nsat
        * b**3
        * (x / b) ** a
        * (c * (x / b) ** c - a + 3)
        * np.exp(-((x / b) ** c))
    )
    bot = x**4
    return -(top / bot)


def finite_difference(func: callable, x: float, h: float) -> float:
    """
    A building block to compute derivative using finite differences
```

```python

    Parameters
    ----------
    func : callable
        Function to differentiate
    x : float
        Value(s) to evaluate derivative at
    h : float
        Step size for finite difference

    Returns
    -------
    dy : float
        Derivative at x
    """
    # return central difference
    return (func(x + h) - func(x - h)) / (2 * h)


def compute_derivative(
    func: callable,
    x: float,
    h_init: float,
    # For Ridders use parameters below:
    d: float = 2.0,
    eps: float = 1e-9,
    max_iters: int = 5,
) -> float:
    """
    Function to compute derivative

    Parameters
    ----------
    func : callable
        Function to differentiate
    x : float
        Value(s) to evaluate derivative at
    h_init : float
        Initial step size for finite difference
    d : float
        Factor by which to decrease h_init every iteration, default = 2.0
    eps : float
        Relative error, default at 1e-9
    max_iters : int
        Maximum number of iterations before exiting, default = 5

    Returns
    -------
    df : float
        Derivative at x
    """
    # local repo
    h = h_init
    d_mat = np.zeros((max_iters, max_iters))
    err_last = np.inf
    best_estimate = d_mat[0, 0]

    # fill 1st col
    for i in range(max_iters):
        d_mat[i, 0] = finite_difference(func, x, h)
        h /= d

```

```python
    # fill the rest
    for j in range(1, max_iters):
        for i in range(max_iters - j):
            top = d ** (2 * j) * d_mat[i + 1, j - 1] - d_mat[i, j - 1]
            bot = d ** (2 * j) - 1
            d_mat[i, j] = top / bot

        # get err estimate
        err = abs((d_mat[0, j] - d_mat[0, j - 1]) / d_mat[0, j])
        if err < eps:
            return d_mat[0, j]
        if err > err_last * 1.2 and j > 1:
            return best_estimate
        # update
        best_estimate = d_mat[0, j]
        err_last = err

    return best_estimate


# DIFF - END
# ================================================================================ #
# MAIN - START


def main():

    # Values from the hand-in
    a = 2.4
    b = 0.25
    c = 1.6
    Nsat = 100
    bounds = (0, 5)
    xmin, xmax = 10 ** (-4), 5
    N_generate = 10000
    xx = np.linspace(xmin, xmax, N_generate)

    def integrand(x: float, a: float = a, b: float = b, c: float = c):
        """Integrand function to replace original lambda fn call"""
        if x != 0.0:
            return 4.0 * np.pi * x**2 * (x / b) ** (a - 3) * np.exp(-((x / b) ** c))
        else:
            return 0.0

    integral, err = romberg_integrator(
        integrand, bounds, order=10, args=(a, b, c), err=True
    )

    # Normalisation
    A = 1.0 / integral
    with open("./output/a2q1_satellite_A.txt", "w") as f:
        f.write(f"{A:.12g}\n")

    def integrand_nsat(
        x: float,
        a: float = a,
        b: float = b,
        c: float = c,
        A: float = A,
```

```python
        Nsat: int = Nsat,
    ):
        """Integrand with Nsat"""
        if x != 0.0:
            return 4.0 * np.pi * x**2 * n(x, A, Nsat, a, b, c)
        else:
            return 0.0

    integral_nsat = romberg_integrator(integrand_nsat, bounds, order=10, err=False
        )

    def p_of_x(x: float):
        """Get p(x)"""
        return integrand(x) / integral

    p_of_x_norm = p_of_x

    random_samples = sampler(
        p_of_x_norm, min_val=xmin, max_val=xmax, Nsamples=N_generate, args=()
    )

    edges = 10 ** np.linspace(np.log10(xmin), np.log10(xmax), 51)
    counts, _ = np.histogram(random_samples, bins=edges)
    bin_widths = np.diff(edges)
    bin_centers = 0.5 * (edges[:-1] + edges[1:])

    hist_scaled = (counts / bin_widths) / (N_generate / Nsat)

    relative_radius = bin_centers
    analytical_function = np.array([Nsat * p_of_x(x) for x in relative_radius])

    # 1b plot gen
    fig1b, ax = plt.subplots()
    ax.stairs(
        hist_scaled, edges=edges, fill=True, alpha=0.6, label="Satellite galaxies"
    )
    plt.plot(relative_radius, analytical_function, "r-", label="Analytical
        solution")
    ax.set(
        xlim=(xmin, xmax),
        ylim=(1e-3, 1e3),  # you may or may not need to change ylim
        yscale="log",
        xscale="log",
        xlabel="Relative radius",
        ylabel="Number of galaxies",
    )
    ax.legend()
    plt.savefig("./plots/a2q1_my_solution_1b.png", dpi=600)
    plt.close(fig1b)

    # Cumulative plot of the chosen galaxies (1c)
    chosen = choice(random_samples, size=100)
    chosen = sort_array(chosen)

    fig1c, ax = plt.subplots()
    ax.step(chosen, np.arange(100))
    ax.set(
        xscale="log",
        xlabel="Relative radius",
        ylabel="Cumulative number of galaxies",
        xlim=(xmin, xmax),
        ylim=(0, 100),
```

```python
    )
    plt.savefig("./plots/a2q1_my_solution_1c.png", dpi=600)
    plt.close(fig1c)

    plot_rng_uniformity()

    x_to_eval = 1
    func_to_eval = lambda x: n(x, A, Nsat, a, b, c)

    dn_dx_numeric = compute_derivative(func_to_eval, x_to_eval, h_init=0.1)
    dn_dx_analytic = dn_dx(x_to_eval, A, Nsat, a, b, c)

    with open("./output/a2q1_satellite_deriv_analytic.txt", "w") as f:
        f.write(f"{dn_dx_analytic:.12g}\n")
    with open("./output/a2q1_satellite_deriv_numeric.txt", "w") as f:
        f.write(f"{dn_dx_numeric:.12g}\n")


# MAIN - END
# ==========================================================================================
#

if __name__ == "__main__":
    main()
```

B Full Code Listing for Question 2

```python
"""
Scripts for assignment 2 question 2
"""

import numpy as np
import matplotlib.pyplot as plt

# ==========================================================================================
# EQFN - START

PSI = 0.929
TC = 1e4  # K
Z = 0.015
K = 1.38e-16  # erg K^(-1)
AB = 2e-13  # cm^(3) s^(-1)
A = 5e-10  # erg
XI = 1e-15  # s^(-1)


def equilibrium1(t: float, z: float = Z, tc: float = TC, psi: float = PSI) -> float:
    """Equilibrium fn 1 for photonization heating and recombination - simple case"""
    # simple case
    # fn = Gamma - Lambda
    return (psi * tc * K) - ((0.684 - 0.0416 * np.log(t / (1e4 * z * z))) * t * K)


def equilibrium2(
    t: float,
    nh: float,
    z: float = Z,
    tc: float = TC,
    psi: float = PSI,
    a: float = A,
    xi: float = XI,
    ab: float = AB,
) -> float:
    """Equilibrium fn 2 for photonization heating and recombination - complex case"""
    # full eq with other factors
    # fn = (Gamma_pe + Gamma_cr + Gamma_mhd) - (Lambda_rr + Lambda_ff)
    return (
        (
            # Gamma_pe
            psi * tc
            # Lambda_rr
            - (0.684 - 0.0416 * np.log(t / (1e4 * z * z))) * t
            # Lambda_ff
            - 0.54 * (t / 1e4) ** 0.37 * t
        )
        * K
        * nh
        * ab
        # Gamma_cr
        + a * xi
        # Gamma_mhd
        + 8.9e-26 * (t / 1e4)
```

```python
    )


# EQFN - END
# ==========================================================================================
# ROOT - START


def root_finder(
    func: callable,
    bracket: tuple[float, float],
    atol: float = 1e-6,
    rtol: float = 1e-6,
    max_iters: int = 1000,
) -> tuple[float, float, float]:
    """
    Find a root of a function - Brent's method

    Parameters
    ----------
    func : callable
        Function fn to find root of
    bracket : tuple
        Bracket for which to find root
    atol : float, optional
        Absolute tolerance.
        The default is 1e-6
    rtol : float, optional
        Relative tolerance.
        The default is 1e-6
    max_iters: int, optional
        Maximum number of iterations.
        The default is 100

    Returns
    -------
    root : float
        Approximate root
    aerr : float
        Absolute error - final bracket width
    rerr : float
        Relative error - final width / |fb|
    """
    # local repo
    a, b = bracket
    fa, fb = func(a), func(b)

    # ensure at least one root is present in bracket
    # ideally only one root present is the best case
    if fa * fb >= 0.0:
        raise ValueError(f"Cannot guarantee n_root >= 1 with set bracket {bracket}.")

    # set b as the best estimate between a and b
    # must have abs(fa) > abs(fb)
    # no swap needed if equal scale
    if abs(fa) < abs(fb):
        a, b = b, a
        fa, fb = fb, fa

```

```python
    # set initial c as a
    # at next iter c is current b
    c = a
    niter = max_iters

    # init new point and diff size
    d = 0.5 * (b - a)
    e = d

    # make current state
    state = (a, b, c, d, e, niter)

    def check_bracket(state: tuple) -> tuple:
        """Check if the current [c, b] bracket is still valid"""
        # unpack and get val
        a, b, c, d, e, niter = state
        fb, fc = func(b), func(c)
        # check bracket
        if fb * fc > 0.0:
            e = b - a
            # fallback to c = a to ensure valid bracket
            state = (a, b, a, d, e, niter)

        return state

    def check_best_estimate(state: tuple) -> tuple:
        """Best estimation should be x=b"""
        a, b, c, d, e, niter = state
        fb, fc = func(b), func(c)

        # set b as best estimate
        # if |fb| > |fc| -> c is better estimate now
        if abs(fb) > abs(fc):
            # set a_new -> b, b_new -> c, c_new -> a
            state = (b, c, a, d, e, niter)

        return state

    def find_new_point(state: tuple) -> tuple:
        """Calculate d from current state"""
        a, b, c, d, e, niter = state
        fa, fb, fc = func(a), func(b), func(c)

        # method select
        if c == a:
            # secant method
            d_new = b - fb * (b - a) / (fb - fa)
        else:
            # quadratic
            r, s, t = fb / fc, fb / fa, fa / fc
            p = s * (t * (r - t) * (c - b) - (1.0 - r) * (b - a))
            q = (t - 1.0) * (r - 1.0) * (s - 1.0)
            d_new = b + p / q

        state = (a, b, c, d_new, e, niter)

        return state

    def check_d(state: tuple) -> tuple:
        """Check if d satisfy the conditions"""
        a, b, c, d, e, niter = state
        low = 0.25 * (3 * a + b)
```

```
        high = b
        h = abs(d - b)
        h_last = abs(e)
        # bisection conditions check
        if (low <= d <= high) and (h < 0.5 * h_last):
            # no bisection
            d_new = d
            e_new = d - b
        else:
            # use bisection
            d_new = 0.5 * (a + b)
            e_new = b - a

        return (a, b, c, d_new, e_new, niter)

    def are_we_there_yet(state: tuple) -> tuple:
        """Check if a root is found, otherwise update state"""
        a, b, c, d, e, niter = state
        fb, fd = func(b), func(d)

        # exit if new point is root
        if fd == 0.0:
            return state

        # update bracket
        if fb * fd < 0.0:
            a_new, b_new = b, d
        else:
            a_new, b_new = a, d

        # enforce b as the best estimate
        fa_new, fb_new = func(a_new), func(b_new)
        # if |fb| > |fa| -> a is better estimate -> swap
        if abs(fb_new) > abs(fa_new):
            a_new, b_new = b_new, a_new

        # update c_new = b
        c_new = b

        # update niter counter
        niter -= 1

        # rebuild state
        state = (a_new, b_new, c_new, d, e, niter)

        return state

    def check_convergence(state: tuple) -> tuple:
        """Recursion stop consition checker"""
        a, b, c, d, e, niter = state
        fb = func(b)

        # get current tolerance
        tol = atol + rtol * abs(b)
        aerr = abs(b - a)
        rerr = aerr
        flag = False
        root = None

        # check convergence
        if (aerr < tol) or (niter == 0) or (fb == 0.0):
            # yes convergence
```

```
            flag = True
            root = b
            if b != 0.0:
                rerr = aerr / abs(b)

        # build output for main return
        result = (flag, niter, root, aerr, rerr)

        return result

    def evolve(state: tuple) -> tuple:
        """Recursivly do Brent's method to get the final state"""
        flag, niter, root, aerr, rerr = check_convergence(state)

        # exit if convergence achieved
        if flag == True:
            return root, aerr, rerr, max_iters - niter

        # main pipeline
        # validity check
        state = check_bracket(state)
        state = check_best_estimate(state)
        # find new point d
        state = find_new_point(state)
        # check if d is valid
        state = check_d(state)
        # evaluate and check
        state = are_we_there_yet(state)

        # recursive
        return evolve(state)

    return evolve(state)


# ROOT - END
# ==================================================================================
# MAIN - START


def main():

    # fig init
    fig, ax = plt.subplots()

    #### 2a ####
    # Initial bracket
    bracket_2a = (1, 1e7)

    root, aerr, rerr, niter = root_finder(func=equilibrium1, bracket=bracket_2a)

    with open("./output/a2q2_equilibrium_temp_simple.txt", "w") as f:
        f.write(f"{root:.12g} & {aerr:.3e} & {rerr:.3e} & {niter}")

    #### 2b ####
    # Initial bracket
    bracket_2b = (1, 1e15)
    # density
    density = [1e-4, 1, 1e4]
    # temperature array
```

```python
    t_ary = np.logspace(1, 15, 10000)

    for nH in density:

        # make lambda fn for nH assignment
        temp_func = lambda t, nh=nH: equilibrium2(t, nh)

        # results
        root, aerr, rerr, niter = root_finder(func=temp_func, bracket=bracket_2b)

        # write
        if nH == 1e-4:
            with open("./output/a2q2_equilibrium_low_density.txt", "w") as f:
                f.write(f"{root:.12g} & {aerr:.3e} & {rerr:.3e} & {niter}")
        elif nH == 1:
            with open("./output/a2q2_equilibrium_mid_density.txt", "w") as f:
                f.write(f"{root:.12g} & {aerr:.3e} & {rerr:.3e} & {niter}")
        elif nH == 1e4:
            with open("./output/a2q2_equilibrium_high_density.txt", "w") as f:
                f.write(f"{root:.12g} & {aerr:.3e} & {rerr:.3e} & {niter}")

        # get f(t)
        f_t = temp_func(t_ary)

        # plot gen
        ax.loglog(t_ary, f_t, label=f"$n_H = {nH:.0e}$ cm$^{{-3}}$")
        ax.axvline(root, color="black", alpha=0.6)

    ax.set_xlabel("Temperature $T$ K")
    ax.set_ylabel("Equalibrium function evaluation")
    ax.legend()
    plt.savefig("./plots/a2q2_equilibrium_function.png", dpi=600)
    plt.close(fig)


# MAIN - END
# ==========================================================================================
#

if __name__ == "__main__":
    main()
```